Wandering in the Woods
Agile Project Documentation

Software Engineering Final Project

Group 6

Table of Contents

1.0 Introduction
1.1 Purpose
1.2 The Wandering in the Woods System
2.0 Process Model
3.0 Use Cases
3.1 Use Case 1: Start a K–2 Simulation
3.2 Use Case 2: Restart the Simulation
3.3 Use Case 3: Configure a Grid
3.4 Use Case 4: Place Players
3.5 Use Case 5: Run a Simulation Step-byStep
3.6 Use Case 6: Run an Automatic Simulation
3.7 Use Case 7: View Statistics
3.8 Use Case 8: Run Experiments
3.9 Use Case 9: Generate a Graph
4.0 UML Model
4.1 Use Case Diagram
4.2 Deployment Diagram
4.3 Class Diagram
4.4 State Diagram
4.5 Activity Diagram
5.0 Customer Journey Map
6.0 Personas
7.0 UI Mock-up
8.0 Testing Strategy
8.1 Unit Testing
8.2 Integration Testing
8.3 Usability Testing
8.4 Validation Testing
9.0 Future Improvements

1. Introduction

1.1 Purpose

This document describes the design and development of **Wandering in the Woods**, an educational simulation created to help K–8 students learn concepts related to randomness, probability, computational thinking, and data analysis.

The document includes the **Software Requirements Specification (SRS)** and the **software design**, including system architecture, use cases, diagrams, testing strategy, and potential future improvements.

1.2 The Wandering in the Woods System

Wandering in the Woods is a simulation in which players (wanderers) move randomly through a forest represented by a rectangular grid.

Players cannot see or hear each other unless they occupy the same grid cell. The simulation ends when all players meet in the same location.

The system supports **three progressively more complex versions**:

Version	Grade Level	Description
Version 1	K–2	Two players wander randomly on a fixed grid until they meet
Version 2	Grades 3–5	Configurable grid and multiple players with statistics
Version 3	Grades 6–8	Experiment mode with graphs and statistical analysis

The application is implemented using **Python**, with **pygame** providing the graphical interface and **matplotlib** generating statistical graphs.

2.0 Process Model

The project was developed using an **evolutionary / agile development process model**.

This approach was chosen because:

- the system contains multiple versions with increasing complexity
- visual simulation software benefits from iterative prototyping
- feedback from testing can be incorporated quickly

Development progressed in the following stages:

1. Implement the core simulation engine
2. Develop Version 1 with a basic visual interface
3. Extend functionality for Version 2 with configurable grids and statistics
4. Implement Version 3 experimental features and graph generation

5. Refine documentation, usability, and testing

3.0 Use Cases

3.1 Use Case 1: Start a K–2 Simulation

Actor: Student pair

Description:

Students start the program and observe two players wandering randomly until they meet.

Outcome:

A celebration animation appears when the players meet.

3.2 Use Case 2: Restart the Simulation

Actor: Student pair

Description:

Students restart the simulation to observe another random run.

Outcome:

The players return to their starting positions and the counters reset.

3.3 Use Case 3: Configure a Grid

Actor: Student pair

Description:

Students choose the width and height of the grid before starting the simulation.

Outcome:

A customized simulation grid is created.

3.4 Use Case 4: Place Players

Actor: Student pair

Description:

Students select starting positions for each player on the grid.

Outcome:

Players appear in the selected cells before the simulation begins.

3.5 Use Case 5: Run a Simulation Step-by-Step

Actor: Student pair

Description:

Students advance the simulation one step at a time.

Outcome:

Players move one step per button press.

3.6 Use Case 6: Run an Automatic Simulation

Actor: Student pair

Description:

Students activate automatic movement to run the simulation continuously.

Outcome:

Players move automatically until they meet.

3.7 Use Case 7: View Statistics

Actor: Student pair

Description:

Students view statistics across multiple simulation runs.

Displayed statistics include:

- shortest run
- longest run
- average run length
- number of runs

3.8 Use Case 8: Run Experiments

Actor: Middle school students

Description:

Students run multiple simulations automatically to analyze patterns.

Outcome:

Results from many runs are collected and summarized.

3.9 Use Case 9: Generate a Graph

Actor: Middle school students

Description:

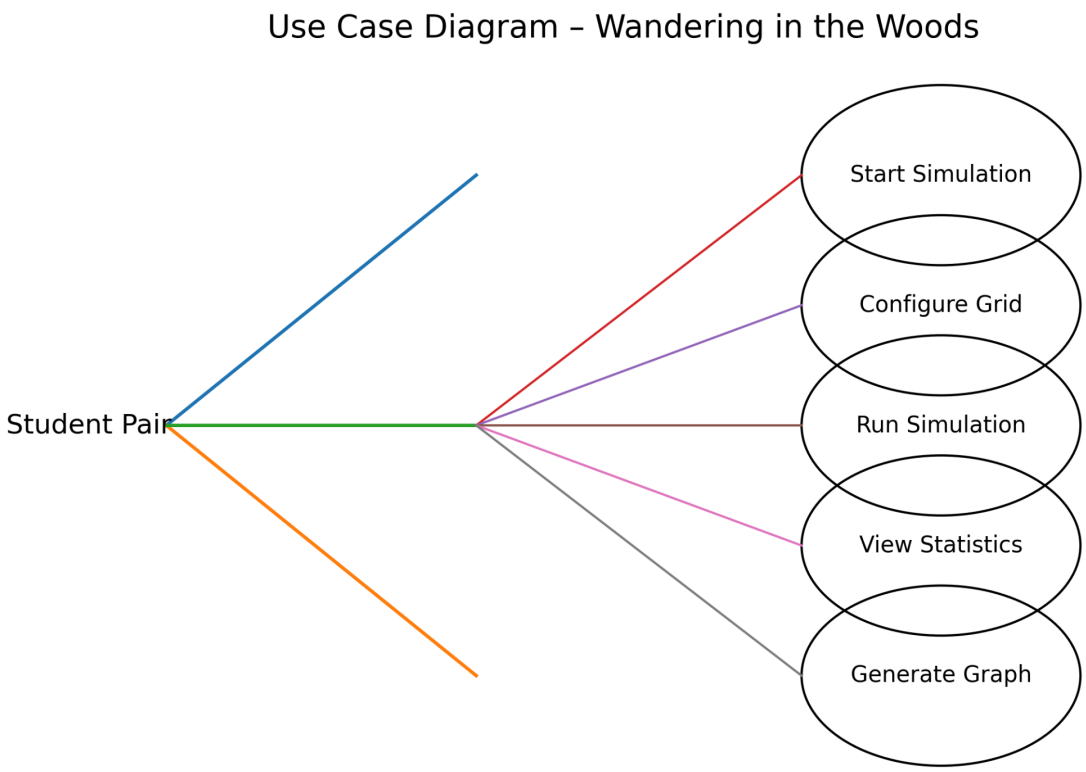
Students generate a histogram showing the distribution of meeting times.

Outcome:

A graph visualizes the simulation results.

4.0 UML Model

4.1 Use Case Diagram



The primary actor is a **student pair** interacting with the simulation.

Major use cases include:

- Start Simulation
- Restart Simulation
- Configure Grid

- Place Players
- Run Simulation
- Step Simulation
- View Statistics
- Run Experiments
- Generate Graph

4.2 Deployment Diagram

The system runs on a **single desktop or laptop computer**.

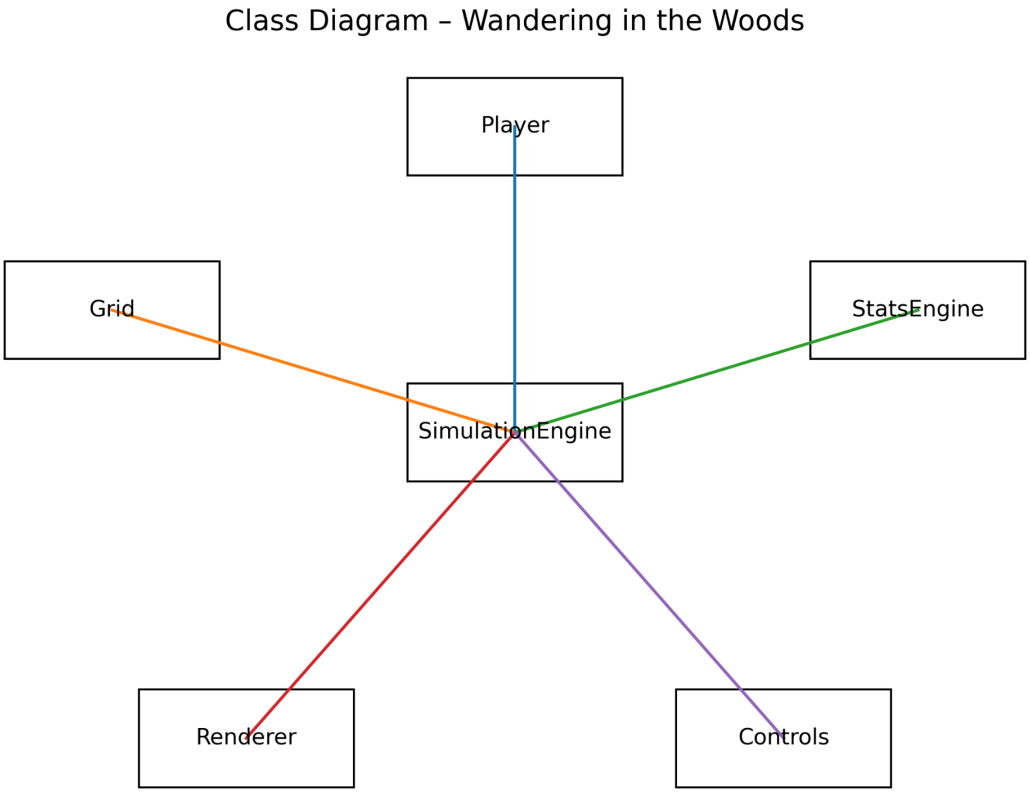
Components include:

- Python runtime
- pygame for graphical interface
- matplotlib for graphs

Users interact through:

- keyboard
- mouse
- display
- speakers (for optional audio prompts)

4.3 Class Diagram



Key classes include:

Grid

- stores grid dimensions
- validates player positions

Player

- stores player ID
- tracks position
- counts steps taken

SimulationEngine

- controls simulation steps
- detects meetings
- merges player groups

Movement Strategy

- defines movement behavior

StatsEngine

- records simulation results

ExperimentRunner

- runs multiple simulations

Renderer

- draws the simulation graphics

Controls

- handles keyboard and mouse input

4.4 State Diagram

Major system states include:

- Program Start
- Setup Screen
- Simulation Running
- Simulation Completed
- Experiment Mode
- Graph Display
- Program Exit

4.5 Activity Diagram

Typical workflow:

1. Launch program
2. Select simulation version
3. Configure grid and players
4. Start simulation

5. Players wander randomly
6. Players meet
7. Statistics recorded
8. Optionally run experiments and generate graphs

5.0 Customer Journey Map

Typical student experience:

1. Start the program
2. Choose a simulation version
3. Configure the simulation
4. Run the simulation
5. Observe the results
6. Restart or modify parameters
7. Compare results across runs
8. Draw conclusions

Possible challenges:

- understanding randomness
- interpreting statistics
- waiting for long runs

6.0 Personas

Persona 1 – Maya (Grade 2 Student)

- enjoys colorful visuals
- prefers simple controls
- learns by observing patterns

Persona 2 – Jordan (Grade 5 Student)

- curious about different outcomes
- interested in experimenting with grid sizes

Persona 3 – Ava (Grade 8 Student)

- interested in analyzing data

- comfortable interpreting graphs

7.0 UI Mock-up

The user interface contains the following main elements:

- simulation grid
- player indicators
- movement counters
- control buttons
- statistics panel
- experiment controls

8.0 Testing Strategy

8.1 Unit Testing

Individual components are tested independently.

Examples:

- player movement
- grid boundary validation
- meeting detection
- statistics calculations

8.2 Integration Testing

Integration tests verify that components work together correctly.

Examples:

- SimulationEngine interacting with Player objects
- GameWindow triggering simulation updates
- statistics updating after simulation completion

8.3 Usability Testing

Usability testing focuses on ensuring the program is easy for students to use.

Key considerations:

- simple controls
- clear visual feedback
- large buttons for younger students

8.4 Validation Testing

Validation testing confirms the system satisfies the project requirements.

This includes verifying:

- Version 1 meets K–2 specifications
- Version 2 supports configurable grids and statistics
- Version 3 supports experiments and graphs

9.0 Future Improvements

Potential enhancements include:

- additional wandering strategies
- heatmap visualization of visited cells
- exporting experiment results to CSV
- enhanced audio instructions
- accessibility improvements
- multiplayer interaction